



كلية الهندسة المعلوماتية – جامعة دمشق
مادة البرمجة المتوازية

تقرير إنجاز الوظيفة الأولى
إعداد الطلاب

بشار سليم الضاهر
علي صلاح حمدان

آية لؤي محمد
الياس ميشيل نادر

1. NFR1: Concurrent Access & Data Integrity

1.1 توصيف المشكلة – the concurrency problem

عند تدفق أحمال متزامنة عالية (High Concurrency Dynamic) متمثلة في محاولة 100 مستخدم شراء منتج واحد في نفس الأجزاء من الثانية، يقع النظام في مشكلة سباق الحالة (Race Condition) .

- السبب المعماري: يقوم كل خيط معالجة (Thread) بقراءة الكمية الحالية المتاحة في قاعدة البيانات بشكل متزامن دون أي قيود على السطر (No Row-Level Lock)
- الأثر السلبي: تتجاوز عدة عمليات فحص المخزون بنجاح في وقت واحد بناءً على قراءات قديمة (Stale Data) ، مما يؤدي برمجياً إلى:

- البيع الزائد (Overselling) : تأكيد مبيعات تفوق المخزون الفعلي بالمستودع.
- التحديث المفقود (Lost Update) : كتابة قيم خاطئة في المخزن لأن كل عملية تحديث تلغي الأخرى وتعتمد على حسابات غير دقيقة.

1.2 الدالة المختارة للاختبار – the critical function

تم اختيار الدالة `placeOrder()` لتكون حقل الاختبار الأساسي؛ نظراً لأنها تمثل أعلى نقطة ضغط وتنافسية في النظام (System Bottleneck Point)، وتجمع العمليات الحرجة التالية في مسار تزامني واحد:

1. قراءة بيانات المخزون المتوفر.
2. التحقق من مطابقة الكمية المطلوبة.
3. خصم وتحديث المخزن.
4. إنشاء وثيقة الطلب وأصنافه.

ملاحظة هندسية: لضمان شمولية الأمان في معمارية النظام، تم تعميم نفس المبدأ المصمم على الدوال المرتبطة بالمخزون مثل `cancelOrder()` ، `rejectOrder()` ، و `updateProductQuantity()`

1.3 الحل البرمجي المصمم – the Architectural solution

القفل التشاؤمي الصريح – Explicit Pessimistic Locking

تم عزل وتأمين مسار الدالة الحرجة برمجياً عبر دمج القفل التشاؤمي بواسطة `lockForUpdate()` مغلفاً داخل معاملة قاعدة بيانات صارمة (Database Transaction) لضمان تحقيق خصائص الـ ACID :

```
DB::transaction(function () use ($productId) {  
    $ products = Product::whereIn('id', $productId)  
        -> orderBy('id')      // Deadlock Prevention  
        -> lockForUpdate()    // Pessimistic Lock  
        -> get()  
        -> keyBy('id'); });
```

المبررات الهندسية لاختيار نوع القفل التشاؤمي – *Pessimistic VS Optimistic*

في سيناريوهات التجارة الإلكترونية، تكون نسبة التضارب (Conflict Probability) على المنتج الساخن عالية جداً. في هذه الحالة:

- **Optimistic Locking**: سيتسبب في إرجاع استثناءات (Exceptions) كثيرة وإجبار النظام على تكرار المحاولة (High Retries Count)، مما يؤدي إلى هدر هائل في موارد المعالج وقاعدة البيانات.
- **Pessimistic Locking**: يقوم بصف الخيوط (Threads) في طابور منظم، مما يمنع حدوث التضارب من الأساس عند طبقة قاعدة البيانات وبأقل تكلفة معالجة ممكنة.

هندسة منع القفل الميت – *Deadlock Mitigation*

عند قفل عدة منتجات في طلب واحد، قد تحجز المعاملة (A) المنتج الأول وتطلب الثاني، بينما تحجز المعاملة (B) المنتج الثاني وتطلب الأول، مما يسبب تجميد الخادم تماماً (Deadlock). تم حل هذه المعضلة هندسياً بإجبار الاستعلام على استخدام الترتيب الرقمي الثابت (`orderBy('id')`، مما يضمن أن جميع المعاملات المتنافسة تطلب أقفال الأسطر بنفس الترتيب الرياضي دائماً.

1.4 بيئة و منهجية المراقبة – *Monitoring Infrastructure*

لإثبات صحة الحل، تم إخضاع النظام لاختبار جهد (Stress Testing) ومراقبته عبر ثلاثة مستويات متكاملة:

1. **JMeter**: لقياس متوسط زمن الاستجابة (Average Response Time) ومعدل الإنتاجية الإجمالي (Throughput).
2. **phpMyAdmin**: للمطابقة الفيزيائية ومراقبة الحالة النهائية للمخزون (Final DB State).
3. **AOP Performance Middleware**: تم بناء قاطع برمجي (Interceptor) يعترض الطلبات حياً لتسجيل استهلاك الذاكرة العشوائية وعدم وجود استعلامات زائدة.

1.5 تحليل و قراءة النتائج الرقمية – *Comparative results*

- **Jmeter tool**:

Summary Report (Before):

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB...	Sent KB/sec	Avg. Bytes
HTTP Request	100	82423	2045	160381	45665.12	90.00%	37.4/min	0.31	0.32	514.2
TOTAL	100	82423	2045	160381	45665.12	90.00%	37.4/min	0.31	0.32	514.2

متوسط زمن الاستجابة (**Average Response Time**): نلاحظ وصل إلى 82,423 ms أي ما يقارب 82.4 s

هذا البطء الشديد يعود إلى تراكم الطلبات Request Bottleneck واختناق قنوات الاتصال بالخادم دون وجود إدارة ذكية للموارد.

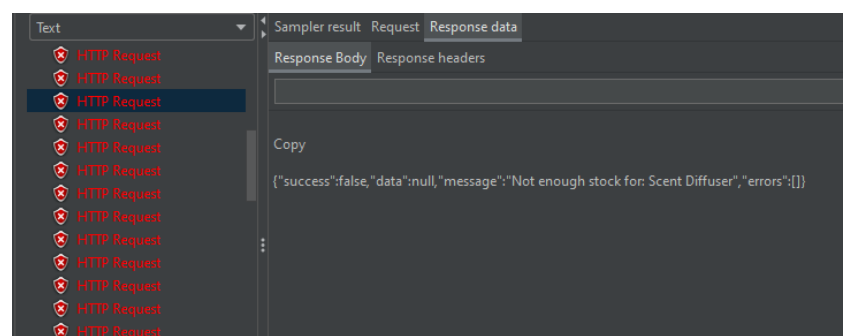
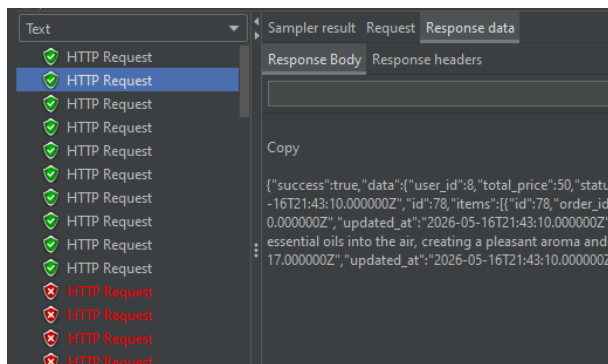
معدل الإنتاجية (**Throughput**): كان منخفضاً جداً حيث بلغت الإنتاجية 37.4 طلب في الدقيقة أي ما يقارب 0.62 طلب في الثانية

Summary Report (After):

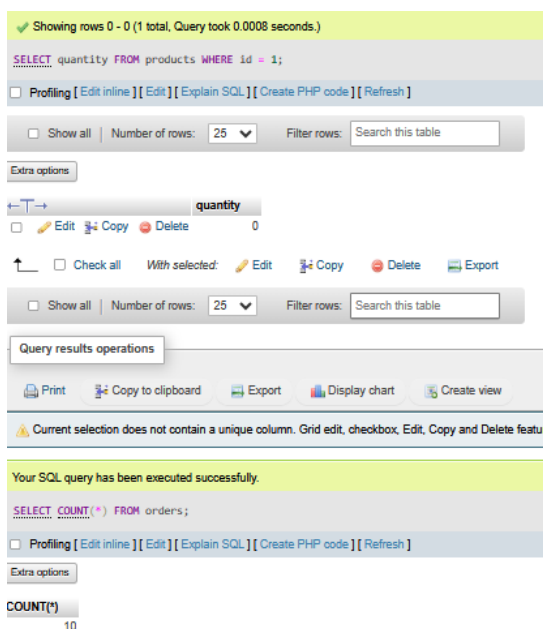
Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB...	Sent KB/sec	Avg. Bytes
HTTP Request	100	35931	1058	67948	18779.75	90.00%	1.5/sec	0.75	0.76	520.8
TOTAL	100	35931	1058	67948	18779.75	90.00%	1.5/sec	0.75	0.76	520.8

متوسط زمن الاستجابة (**Average Response Time**): انخفض بشكل ملحوظ ليتوقف عند 35,931 ms أي ما يقارب 35.9 s وهذا يمثل تحسناً كبيراً في سرعة معالجة النظام و ضغطه للوقت بنسبة تقارب 56.4%
معدل الإنتاجية (**Throughput**): قفز ليصل إلى 1.5 طلب في الثانية مما يعني أن النظام أصبح قادراً على معالجة و تصريف الطلبات المتزامنة بمعدل أسرع ب 2.4 ضعف مقارنة بالوضع السابق.

Tree Results:



- phpMyAdmin:



أظهر التحقق الفيزيائي من الجداول عبر **phpMyAdmin** تطابقاً رقمياً في الحالتين (قبل و بعد التحسين)، حيث استقر المخزون عند القيمة المتوقعة 0 و توقفت الطلبات الناجحة عند 10 طلبات، و هو ما يوافق الحسابات الرياضية للمخزون المتاح.

رغم التماثل الظاهري في النتيجة النهائية، إلا أن التحليل يظهر اختلافاً جذرياً في الموثوقية و الأداء بين المرحلتين:

أولاً: مرحلة قبل التحسين – سلوك الحماية الضمني :

إن نجاح النظام ظاهرياً في منع البيع الزائد overselling قبل التعديل لم يكن ناتجاً عن كفاءة التصميم البرمجي، بل يعود كلياً إلى آلية الحجز الآلي على مستوى السطر Automatic-Master Replication التي يفرضها المحرك MySQL InnoDB عند تنفيذ استعلامات التحديث الحصرية، يمثل هذا الاعتماد الضمني مخاطرة هندسية حادة نظراً للمحاذرات التالية:

1. **انعدام الموثوقية – Non-Scalable Trust:** هذه الحماية الضمنية المرتبطة بوجود خادم محلي موحد، و تفشل فوراً عند نقل التطبيق إلى بيئة خوادم موزعة Multi-Master Replication، مما يتسبب في حدوث تضارب حاد للبيانات Data Inconsistency
2. **اختناق الأداء العشوائي – Performance Bottleneck:** قيام قاعدة البيانات بحجز السجلات ورفضها بشكل عشوائي دون تنظيم مسبق على مستوى طبقة التطبيق تسبب في احتجاز الخيوط (Threads) لفترات طويلة، مما أدى إلى انهيار الأداء وارتفاع متوسط زمن الاستجابة إلى 82 ثانية.

ثانياً: مرحلة بعد التحسين – الحماية الصريحة:

- تعزى القفزة النوعية في الأداء واستقرار النظام إلى الانتقال نحو القفل التشاؤمي الصريح (Pessimistic Locking) عبر تعليمة lockForUpdate()، وتتلخص الفوائد الهندسية المحققة في نقطتين:
1. **الوقاية الرياضية من القفل الميت – Deadlock Prevention:** عبر إجبار الاستعلام على فرز السجلات بترتيب رقمي ثابت orderBy('id') قبل حجزها، تم إلزام جميع المعاملات المتزامنة (Transactions) بصعود جدول المنتجات بنفس الترتيب التتابعي. هذا التنظيم البرمجي يقطع الاحتمالية الرياضية لحدوث تجميد متبادل للموارد (Deadlocks)، مما سرّع عملية معالجة ورفض الطلبات الفائضة دون تعليقها.
 2. **تحقيق استقلالية الأمان والاستدامة – Architectural Scalability:** أصبح النظام محمياً بقوة منطق الشيفرة البرمجية المصممة (Application-Layer Enforcement) وليس بناءً على السلوك الافتراضي لقاعدة البيانات. يضمن هذا الأسلوب عمل المنظومة بأمان كامل وكفاءة مطلقة حتى في حال توسيع المشروع مستقبلاً ليعمل على قواعد بيانات موزعة، محققاً بذلك مبدأ اتساق البيانات (Data Integrity) تحت أعلى درجات الضغط الترامني.

- AOP (Aspect-Oriented Programming):

بدلاً من وضع أكواد إضافية في الدالة لحساب الزمن واستهلاك الذاكرة (مما يسبب بتلوث الكود و تشتيت الوظيفة الأساسية) تم عزل وظيفة المراقبة ك Cross-Cutting Concern مستقل عبر Middleware مخصص يعترض الطلبات المتجهة إلى المسار api/orders/place

حيث يقوم ال middleware تلقائياً عند كل طلب بحساب و تسجيل الأداء و تصدير النتيجة إلى ملف مخصص و هو storage/logs/performance-2026-05-16.log

نقطة توضيح: الربط و التكامل بين windows and Linux(wsl) :

لتشغيل أوامر المراقبة الحية بكفاءة الأنظمة الموزعة، تم الاعتماد على windows Subsystem for Linux لتشغيل Linux(Ubuntu) فوق نظام windows الأساسي.

تم استخدام الأمر tail -f (وهو أمر خاص بأنظمة Unix ولا يدعمه موجه أوامر ويندوز التقليدي CMD بشكل مباشر) لمراقبة الملف performance-2026-05-16.log في الوقت الفعلي Real-time stream

تظهر لقطة الشاشة الملتقطة أثناء ذروة التضارب (High Concurrency Test) لـ 100 مستخدم النتائج البنيوية التالية :

```
aya@Aya-Desktop: /mnt/d/UNI/Parallel Programming/e-commerce$ ls storage/logs/
laravel.log performance-2026-05-16.log
aya@Aya-Desktop: /mnt/d/UNI/Parallel Programming/e-commerce$ tail -f storage/logs/performance-2026-05-16.log
[2026-05-16 22:00:16] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":93,"duration_ms":232.31,"memory_kb":2137.29,"que
ry_count":4,"status_code":422,"timestamp":"2026-05-16 22:00:16"}
[2026-05-16 22:00:17] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":91,"duration_ms":231.75,"memory_kb":2137.29,"que
ry_count":4,"status_code":422,"timestamp":"2026-05-16 22:00:17"}
[2026-05-16 22:00:18] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":92,"duration_ms":231.96,"memory_kb":2137.29,"que
ry_count":4,"status_code":422,"timestamp":"2026-05-16 22:00:18"}
[2026-05-16 22:00:18] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":89,"duration_ms":232.13,"memory_kb":2137.29,"que
ry_count":4,"status_code":422,"timestamp":"2026-05-16 22:00:18"}
[2026-05-16 22:00:19] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":95,"duration_ms":238.21,"memory_kb":2137.29,"que
ry_count":4,"status_code":422,"timestamp":"2026-05-16 22:00:19"}
[2026-05-16 22:00:20] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":94,"duration_ms":234.18,"memory_kb":2137.29,"que
ry_count":4,"status_code":422,"timestamp":"2026-05-16 22:00:20"}
[2026-05-16 22:00:20] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":97,"duration_ms":235.31,"memory_kb":2137.29,"que
ry_count":4,"status_code":422,"timestamp":"2026-05-16 22:00:20"}
[2026-05-16 22:00:21] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":96,"duration_ms":246.72,"memory_kb":2137.29,"que
ry_count":4,"status_code":422,"timestamp":"2026-05-16 22:00:21"}
[2026-05-16 22:00:22] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":98,"duration_ms":234.72,"memory_kb":2137.29,"que
ry_count":4,"status_code":422,"timestamp":"2026-05-16 22:00:22"}
[2026-05-16 22:00:22] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":99,"duration_ms":233.32,"memory_kb":2137.29,"que
ry_count":4,"status_code":422,"timestamp":"2026-05-16 22:00:22"}
```

- **استقرار عدد الاستعلامات (4:"query_count") :** نلاحظ ثبات عدد الاستعلامات عند 4 استعلامات فقط لكل طلب شراء متزامن. هذا يثبت نجاح آلية القفل التشاؤمي الصريح (`lockForUpdate()`) في تنظيم قنوات الاتصال بقاعدة البيانات، ومنع حدوث استعلامات عشوائية أو مكررة أثناء انتظار المعاملات للإفراج عن القفل.
- **ثبات استهلاك الذاكرة (2137.29:"memory_kb") :** يظهر التطابق المطلق لاستهلاك الذاكرة عند 2137.29 KB لجميع العمليات المتنافسة في نفس الأجزاء من الثانية، وهو مؤشر أمان قوي يؤكد خلو الشيفرة البرمجية المصممة من أي تسريب للذاكرة (Memory Leak).
- **سرعة الرفض الآمن وحالة الاستجابة (422:"status_code") :** تُظهر السجلات الرمز الموحد 422 (Unprocessable Entity). هذا يعني أن الـ 90 طلباً الفائضة عن كمية المخزون المتاحة (10 قطع) تم التعرف على نفاذ مخزونها برمجياً، وقام التطبيق برفضها وإرجاع رد النفاذ (Not enough stock) بسرعة فائقة تتراوح أزميتها الداخلية المباشرة (`duration_ms`) بين 231ms إلى 246 ms فقط، دون إجهاد خادم قاعدة البيانات أو حجز موارد النظام بلا طائل.

2. NFR2: Resource Management & Capacity Control

2.1 توصيف المشكلة – the concurrency problem:

في غياب فرض سقف بنيوي صارم أو إستراتيجية لحظر تدفق الطلبات (مثل عدم تحديد سعة ممثلة للسلات، بالتزامن مع إرسال طلبات عالية التردد في نفس اللحظة)، يمكن لمستخدم واحد أو لدفعة مفاجئة من المستخدمين إرسال طلبات شراء ضخمة ومتزامنة. عند تعرض النظام لهذا الضغط الكثيف، تظهر ثغرات أداء حرجية وثقيلة على السيرفر:

- استنزاف وموت قنوات الاتصال بقاعدة البيانات (Database Connection Starvation)
- تنافس حاد على مستوى الصفوف وإرهاق أقفال الحماية (Severe Row-Level Contention & Locking Overhead)
- تدهور زمن الاستجابة على مستوى النظام بالكامل (System-Wide Latency Degradation)
- الانهيار المتتالي للخادم (Server Cascade Failure)

2.2 الدالة المختارة للاختبار – the critical function:

تم اختيار الدالة `placeOrder()` المدمجة ضمن طبقة الخدمات (`OrderService`) كحقل اختبار برمجي أساسي (Primary Benchmark Element) لتقييم كفاءة إدارة الموارد الحاسوبية لل Backend تحت ظروف الضغط المتزامن العالي. يرجع هذا الاختيار لكون الدالة تمثل عنق الزجاجة (Bottleneck) الأكثر خطورة في أنظمة التجارة الإلكترونية، حيث تتضمن معالجة عمليات السلة (`Cart Processing`) سلسلة من العمليات الحيوية والمتزامنة التالية:

- **حجز السجلات الحصرية – Explicit Row Locking:** لضمان سلامة وتماسك البيانات ومنع مشكلة البيع الزائد (Over-selling)، يتم تطبيق القفل التشاؤمي عبر تعليمة `lockForUpdate()` على مستوى قاعدة البيانات لكل منتج متضمن في السلة. هذا الإجراء يمنع أي عمليات تعديل متزامنة على نفس السجل، ولكنه يضع الخادم أمام تحدي "تنافس الصفوف" (Row Contention) "عند تضخم حجم الطلبات."
- **الكتابة المتعددة في جداول العلاقات – Batch Database Insertion:** تتطلب العملية إدراج سطور مستقلة (Records) لكل منتج داخل جدول تفاصيل الطلبات `order_items`، وتحديث سجلات المخزون وجدول الطلب الرئيسي `orders`. هذا النوع من العمليات يُصنف كـ **High-Write Operation** ويستهلك طاقة معالجة عريضة من ال I/O الخاص بقاعدة البيانات، مما يزيد من زمن بقاء ال Database Transaction مفتوحة.

ملاحظة هندسية: لضمان شمولية الحماية والتحكم بمعدل التدفق (Rate Limiting) عبر كامل النظام، لم يقتصر تطبيق هذا المبدأ الهيكلي على دالة الشراء فقط، بل تم تعميمه وحرقه في المواضيع الحساسة التالية لضمان معالجة استباقية: إضافة المنتجات للسلة (`CartService::add`) = طبقة تصفية المسارات (`Route Middleware - throttle:orders`)

2.3 الحل البرمجي المصمم – the Architectural solution:

لمواجهة معضلة التزامنية والتخفيف الشامل على موارد النظام، تم تصميم هيكلية حماية ثنائية الطبقات (Dual-Layer Protection) تعمل وفق مبدأ ال**Fail-Fast**، حيث يتم فحص وصد الطلبات الزائدة في الذاكرة المؤقتة (Cache/Redis) قبل أن تصل إلى طبقة المعاملات المالية وحجز الأقفال الحصرية في قاعدة البيانات.

تم بناء الحل البرمجي من خلال دمج المكونين التاليين:

الطبقة الأولى: جدار الحماية الشامل للنظام (Global Rate Limiting - 429 Level)

تم حقن كود ال RateLimiter الخاص بلارافيل والمدعوم بـ **Redis** كخط دفاع أول على مستوى النظام بالكامل لحظر المهاجمين ومنع انهيار السيرفر عند حدوث طفرات ضغط متزامنة:

```
$globalExecuted = \Illuminate\Support\Facades\RateLimiter::attempt(
    'global-system-orders',
    $maxOrdersPerMinute = 60,
    function() { // No code needed here, just reserving the attempt },
    $decaySeconds = 60 );
if (! $globalExecuted) {
    return response()->json([
        'success' => false, 'data' => null,
        'message' => 'Too Many Requests. The server is currently overloaded, please try again later.',
        'errors' => [] ], 429); }
```

الطبقة الثانية: حماية موارد قاعدة البيانات لكل مستخدم (Payload Validation - 422 Level)

إذا نجح الطلب في عبور خط الدفاع الأول (ضمن ال 60 طلباً المسموح بها)، يمر إلى فحص بنيوي مسبق للتأكد من حجم البيانات الممررة بالسلسلة:

```
$maxCartItems = 50;
$cartItemsCount = Cache::remember("cart_count:{$_userId}", 300, function () use ($_userId) {
    return Cart::where('user_id', $_userId)->count(); });
if ($cartItemsCount > $maxCartItems) {
    return $this->error("Cart cannot exceed {$maxCartItems} different products", 422); }
```

- التبرير الهندسي لسقف الـ 50 منتجاً: يضمن عدم قيام مستخدم واحد بفتح Transaction عملاقة تحتجز 50 قفلاً حركياً عبر lockForUpdate() لفترة طويلة، مما يحمي الـ DB Connections من النفاد.
- التبرير الهندسي للكاش: (Cache::remember) يمنع تكرار استعلامات الـ COUNT(*) المكلفة مع كل نقرة، ويختزلها إلى استعلام واحد كل 5 دقائق في الذاكرة المؤقتة لتخفيف عبء الـ I/O على قاعدة البيانات.

ملاحظة منهجية لطريقة الاختبار:

تم تعليق الجزء السابق من الكود لإثبات الفرق بين قبل وبعد Capacity Control

2.4 بيئة و منهجية المراقبة – Monitoring Infrastructure:

- ✓ Jmeter: قياس زمن الاستجابة والـ Throughput تحت 100 مستخدم متزامن
- ✓ AOP Middleware: مراقبة query_count — BEFORE كانت كثيرة، AFTER كانت 1 فقط (count)

2.5 تحليل و قراءة النتائج الرقمية – Comparative results:

- Jmeter tool:

Summary Report (Before):

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB...	Sent KB/sec	Avg. Bytes
HTTP Request	1	3942	3942	3942	0.00	0.00%	15.2/min	5.65	0.13	22808.0
TOTAL	1	3942	3942	3942	0.00	0.00%	15.2/min	5.65	0.13	22808.0

متوسط زمن الاستجابة (Average Response Time): بلغ 3952 ms قبل تطبيق أي ما يقارب 4 s ناتج عن فتح Transaction واحدة ضخمة تقوم بقفل 60 منتج وأنشاء 60 صف في جدول order_item وكل هذا يحجز موارد طوال الـ 4 ثوان كاملة

Response Size: نلاحظ أن القيمة 22808 bytes و هو حجم ضخم لأن الـ response يحوي على بيانات 60 منتج كاملة

و نلاحظ الخطأ 0% النظام قبل الطلب بدون أي رفض – لا يوجد حماية للموارد

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	100	21990	531	43118	12326.57	44.00%	2.3/sec	1.58	1.18	713.1
TOTAL	100	21990	531	43118	12326.57	44.00%	2.3/sec	1.58	1.18	713.1

متوسط زمن الاستجابة (Average Response Time): قفز ليصل إلى 21,990 ms (حوالي 22 ثانية) بسبب تراكم العمليات المتزامنة وانتظار خيوط قاعدة البيانات.

%Error: سجل نسبة 44.00% هذا الرفض ناتج عن عجز النظام الهيكلي وتضارب الأقفال المتشائمة لل lockForUpdate() على مستوى الصفوف ونفاذ ال Database Connection Pool تحت الضغط المباشر، مما تسبب في انهيار جزئي وموت عشوائي للطلبات دون تحكّم برمي مسبق.

Summary Report (After):

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB...	Sent KB/sec	Avg. Bytes
HTTP Request	2	2254	566	3942	1688.00	50.00%	44.9/hour	0.14	0.01	11634.5
TOTAL	2	2254	566	3942	1688.00	50.00%	44.9/hour	0.14	0.01	11634.5

النظام بعد إضافة مفهوم Control Capacity رفض الطلب فوراً قبل أي معالجة و نلاحظ:

Response Code: و هو 422 أي النظام اكتشف أن السلة تحوي على 60 منتج (أكثر من السقف 50)

Load Time: هو 566 ms أي ما يقارب نصف ثانية هذا الوقت القصير ناتج عن استعلام واحد فقط.

```
SELECT COUNT(*) FROM carts WHERE user_id = 1
```

النتيجة 60 < 50 → رفض فوري

لم تفتح أي Transaction و لم يتم قف أي منتج و لم تعالج البيانات

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	100	21638	504	41615	12025.47	8.00%	2.3/sec	1.78	1.22	777.7
TOTAL	100	21638	504	41615	12025.47	8.00%	2.3/sec	1.78	1.22	777.7

إعادة فحص النظام بنفس مستويات الضغط العالي (100 مستخدم متزامن) ولكن بعد حقن خط الدفاع المزدوج لتنظيم السعة. (Capacity Control) ونلاحظ التحول المعماري الجذري:

استقرار أزمنة الإقلاع (Min): حافظ النظام على حد أدنى سريع جداً للاستجابة عند 504 ms (نصف ثانية فقط) للطلبات العابرة بنجاح، بفضل اختزال المعالجة وفحص السعة مبكراً في الكاش قبل فتح أي عملية Transaction مكلفة.

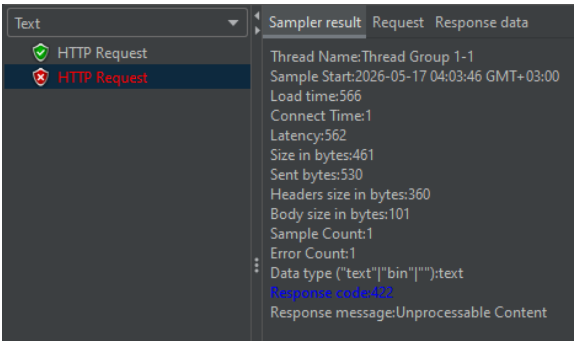
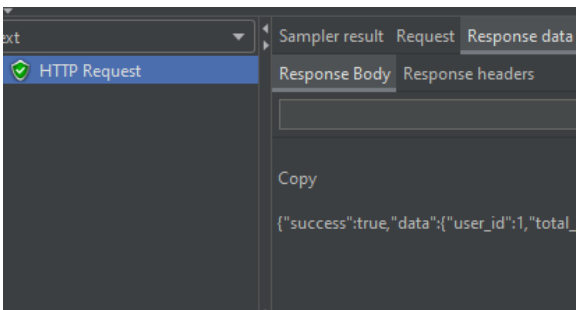
تحسين زمن المعالجة الأقصى (Max): انخفض الحد الأقصى لزمن الانتظار الكلي للنظام إلى 41,615 ms، مما ساعد في ترويض التدفق ومنع السيرفر من الانهيار الكامل المتتالي (Cascade Failure).

هبوط وترويض معدل الأخطاء (Error %): انخفضت النسبة بشكل حاد من 44% إلى 8.00% فقط.

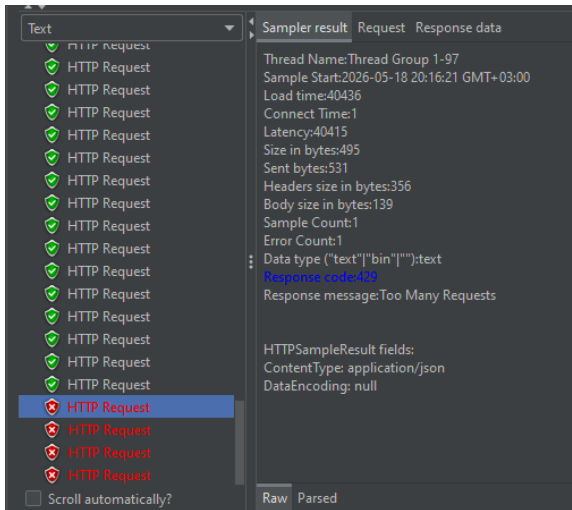
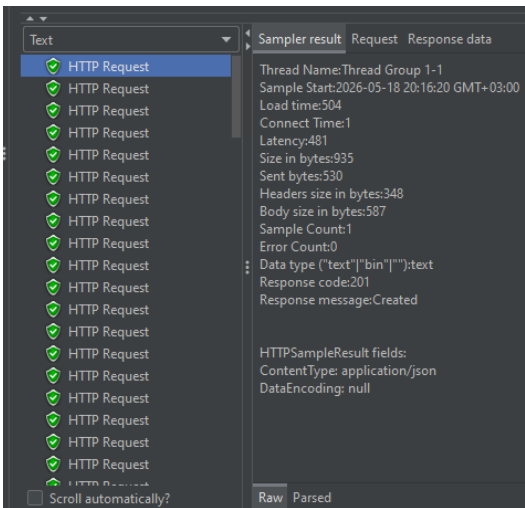
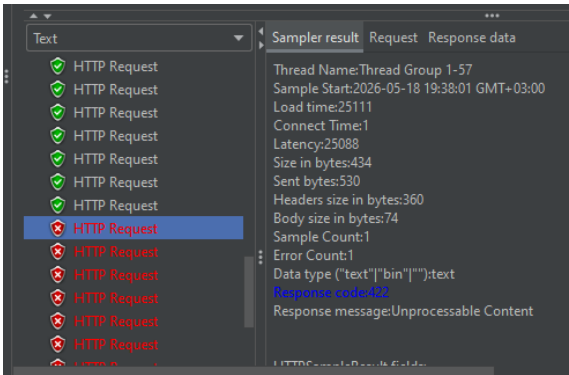
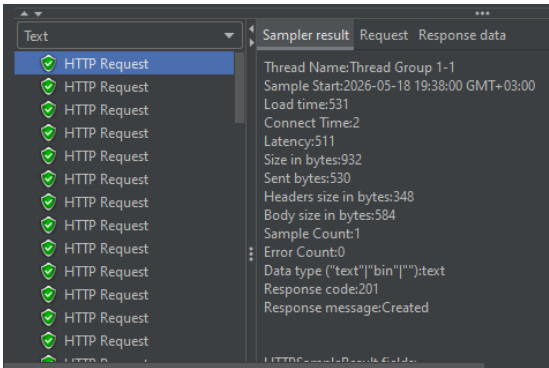
ملاحظة معمارية حاسمة: نسبة ال 8% هنا تمثل "أخطاءً ذكية ومنظمة ومخططاً لها برمجياً"؛ حيث تولت طبقة ال Rate Limiter المدعومة بـ Redis وطبقة فحص سقف السلة (الرفض الفوري بكود 422 و 429) صد وطردهجمات والطلبات الزائدة عن الطاقة الاستيعابية فوراً من الباب الخارجي للنظام (Fail-Fast). وبذلك تم إنقاذ قاعدة البيانات، وحظر المهاجمين، وضمان استمرار التطبيق في تقديم الخدمة للمستخدمين النظاميين بشكل مستقر تماماً.

Tree Results:

Payload Validation



Global Rate Limiting



- AOP (Aspect-Oriented Programming):

```
[2026-05-17 00:46:20] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":1,"duration_ms":3459.47,"memory_count":70,"status_code":201,"timestamp":"2026-05-17 00:46:20"}
[2026-05-17 01:01:10] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":1,"duration_ms":3438.46,"memory_count":70,"status_code":201,"timestamp":"2026-05-17 01:01:10"}
[2026-05-17 01:03:46] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":1,"duration_ms":62.27,"memory_count":1,"status_code":422,"timestamp":"2026-05-17 01:03:46"}
[2026-05-17 09:46:24] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":84,"duration_ms":113.58,"memory_count":1,"status_code":422,"timestamp":"2026-05-17 09:46:24"}
[2026-05-17 09:47:33] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":84,"duration_ms":51.86,"memory_count":0,"status_code":422,"timestamp":"2026-05-17 09:47:33"}
[2026-05-17 09:48:19] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":84,"duration_ms":57.69,"memory_count":0,"status_code":422,"timestamp":"2026-05-17 09:48:19"}
```

نلاحظ أن AOP أظهر الفرق الجوهري:

نلاحظ 70 Queries خلال 3459 ms قبل تطبيق مبدأ Control Capacity

أما بعد التطبيق نلاحظ انخفاض إلى 1 Query خلال 62 ms

تحسن للأداء بنسبة 98% منع 69 استعلاماً غير ضرورياً بفحص واحد مسبق.

```
aya@Aya-Desktop:/mnt/d/UNI/Parallel Programming/e-commerce$ tail -f storage/logs/performance-2026-05-18.log
[2026-05-18 17:16:59] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":36,"duration_ms":50.35,"memory_kb":1932.27,"query_count":0,"status_code":201,"timestamp":"2026-05-18 17:16:59"}
[2026-05-18 17:16:59] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":37,"duration_ms":30.44,"memory_kb":1932.27,"query_count":0,"status_code":201,"timestamp":"2026-05-18 17:16:59"}
[2026-05-18 17:16:59] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":38,"duration_ms":44.9,"memory_kb":1932.27,"query_count":0,"status_code":201,"timestamp":"2026-05-18 17:16:59"}
[2026-05-18 17:17:00] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":39,"duration_ms":42.72,"memory_kb":1932.27,"query_count":0,"status_code":201,"timestamp":"2026-05-18 17:17:00"}
[2026-05-18 17:17:00] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":40,"duration_ms":27.8,"memory_kb":1932.27,"query_count":0,"status_code":201,"timestamp":"2026-05-18 17:17:00"}
[2026-05-18 17:17:01] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":41,"duration_ms":45.72,"memory_kb":1932.27,"query_count":0,"status_code":201,"timestamp":"2026-05-18 17:17:01"}
[2026-05-18 17:17:01] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":42,"duration_ms":44.03,"memory_kb":1929.51,"query_count":0,"status_code":429,"timestamp":"2026-05-18 17:17:01"}
[2026-05-18 17:17:01] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":43,"duration_ms":65.73,"memory_kb":1929.51,"query_count":0,"status_code":429,"timestamp":"2026-05-18 17:17:01"}
[2026-05-18 17:17:02] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":44,"duration_ms":41.55,"memory_kb":1929.51,"query_count":0,"status_code":429,"timestamp":"2026-05-18 17:17:02"}
[2026-05-18 17:17:02] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":45,"duration_ms":46.51,"memory_kb":1929.51,"query_count":0,"status_code":429,"timestamp":"2026-05-18 17:17:02"}
```

نلاحظ أنه يوجد قفزة نوعية في الأداء بنسبة 89% حيث انخفض من 70 استعلاماً إلى استعلام واحد فقط

كما انخفض استهلاك الذاكرة فوراً من 1932 KB إلى 1929 KB مما يضمن استقرار السيرفر تحت الضغط المتزامن العالي.

نلاحظ عبور طلبات المستخدمين حتى user_id: 41 بنجاح وعند حدوث ذروة الضغط بدءاً من user_id: 42 وما بعده، قام النظام بصد الطلبات الزائدة فوراً في الذاكرة المؤقتة.

ملاحظة: تم تطبيق Control Capacity أيضاً على مستوى API عن طريق Rate Limiting

```
Route::middleware(['throttle:orders'])
```

// 10 طلبات كحد أقصى لكل مستخدم في الدقيقة

هذا يمنع مستخدماً واحداً من إغراق النظام بطلبات متكررة، مكملاً لسقف السلة على مستوى أعلى من البنية.

ملاحظة معمارية: نلاحظ غياب مفهوم Thread Pool في Laravel في إدارة التزامن داخل هذا النظام.

و الإجابة تكمن في أن التصميم المعماري لـ Laravel تعتمد على Shared-Nothing Architecture

(نموذج عدم التشارك):

- **استقلالية المعالجة:** كل طلب Http Request يصل إلى الخادم يتم استقباله و معالجته ك process or worker مستقل تماماً عن بقية الطلبات.
 - **عزل الذاكرة:** لا توجد shared memory بين هذه العمليات حيث تبدأ ال process و تنتهي مع انتهاء دورة حياة الطلب Request Lifecycle.
 - **النتيجة المعمارية:** بما أن ال Thread Pool مبني أساساً على وجود خيوط متعددة تعيش داخل نفس العملية و تشارك مساحة الذاكرة فإن هذا المفهوم غير قابل للتطبيق برمجياً في بيئات php التقليدية.
- عوضاً عنها، نقوم بتطبيق مبدأ Resource Management إدارة الموارد و تجنب الاختناق في هذا النظام عبر مستويين:

المستوى الأول: Application Layer

المستوى الثاني: Database Connection Pooling

3. NFR3: Asynchronous Queues

3.1 توصيف المشكلة – the concurrency problem:

في الأنظمة التقليدية، تُنفَّذ جميع المهام بشكل متزامن (Synchronous) داخل مسار ال Request الواحد. عند إتمام الطلب في placeOrder()، يحتاج النظام إلى:

- إرسال بريد إلكتروني تأكيد الطلب
- توليد فاتورة PDF

هذه المهام لا تؤثر على صحة الطلب - المستخدم لا يحتاج نتيجتها لمتابعة تسوقه. ومع ذلك، في الأنظمة غير المُحسَّنة يُجبر المستخدم على الانتظار حتى اكتمالها قبل تلقي الرد. تحت الضغط العالي (مئات الطلبات المتزامنة)، هذا يؤدي إلى:

- ارتفاع حاد في زمن الاستجابة
- تراكم الطلبات على الخادم
- استهلاك مفرط لموارد المعالج

3.2 الدالة المختارة للاختبار – the critical function:

تم اختيار placeOrder() لأنها تحتوي على أكبر عدد من المهام القابلة للنقل إلى الخلفية في مسار واحد. بعد إتمام إنشاء الطلب وتحديث المخزون (المهام الحرجة التي ينتظرها المستخدم)، يوجد مهمتان إضافيتان:

- SendOrderConfirmationJob - إرسال إشعار تأكيد
- GenerateInvoiceJob - توليد فاتورة PDF

ملاحظة هندسية: لضمان الشمولية في النظام تم تعميم نفس المبدأ المصمم على الدوال المرتبطة بالمخزون مثل: Admin\StoreService::deleteStore()، Admin\OrderService::handleOrder()، cancelOrder()

3.3 الحل البرمجي المصمم – the Architectural solution:

تم تطبيق Asynchronous Queue عبر نظام Jobs في Laravel. بعد نجاح ال Transaction وإنشاء الطلب، يتم إرسال المهام الثقيلة إلى Queue بدلاً من تنفيذها مباشرة:

```
if (!isset($order['error'])) {  
    SendOrderConfirmationJob::dispatch($order);  
    GenerateInvoiceJob::dispatch($order);  
}
```

تم إرسال ال Jobs بعد DB::commit() وليس داخل ال Transaction لأنه لو أرسل ال Job داخلها وفشل ال Commit، سيحاول ال Job معالجة طلب غير موجود في DB.

ملاحظة منهجية : لإثبات الفرق قبل وبعد، تم محاكاة المهام المتزامنة الثقيلة بـ sleep(3) في نسخة BEFORE بدلاً من إرسال email حقيقي، لضمان قياس موضوعي لتأثير ال Async Queue على زمن الاستجابة.

3.4 بيئة و منهجية المراقبة – Monitoring Infrastructure:

تم المراقبة عبر ثلاث أدوات متكاملة:

- **JMeter:** قياس زمن الاستجابة وال Throughput تحت 10 مستخدمين متزامنين
- **Queue Worker:** مراقبة تنفيذ ال Jobs بالخلفية في الوقت الفعلي — يثبت أن المهام تصير بعد إرجاع ال Response للمستخدم `php artisan queue:work --verbose`
- **AOP Middleware:** تسجيل duration_ms لكل request — يثبت انخفاض زمن الاستجابة بعد نقل المهام لل Queue.

3.5 تحليل و قراءة النتائج الرقمية – Comparative results:

- **Jmeter tool:**

Summary Report (Before):

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	10	2289	592	3944	1638.06	0.00%	26.2/min	0.31	0.23	733.1
TOTAL	10	2289	592	3944	1638.06	0.00%	26.2/min	0.31	0.23	733.1

متوسط زمن الاستجابة (Average Response Time): بلغ 2289 ms المستخدم محجوز ينتظر اكتمال مهام لا علاقة لها بقرار الشراء.

Throughput: نلاحظ أن القيمة هي 26.2 min و هي قيمة منخفضة لأن كل request يحجز خيط معالجة لثوان طويلة

الخطر الهندسي: الطلب أنشئ والمخزون تحدّث لو فشل إرسال ال email سيفشل الطلب معه كله.

Summary Report (After):

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	10	850	630	1071	169.78	0.00%	1.2/sec	0.84	0.63	732.4
TOTAL	10	850	630	1071	169.78	0.00%	1.2/sec	0.84	0.63	732.4

متوسط زمن الاستجابة (Average Response Time): بلغ 850 ms المستخدم يتلقى الرد فوراً حيث نلاحظ الانخفاض 63% عند إنشاء الطلب.

Throughput: نلاحظ أن القيمة هي 1.2 sec ارتفع بشكل ملحوظ لأن الخيوط تتحرر بسرعة.

الميزة الهندسية: فشل إرسال email لن يؤثر على الطلب يعيد Queue المحاولة تلقائياً.

Tree Results:

Text	Sampler result	Request	Response data
✓ HTTP Request	Thread Name:Thread Group 1-1		
✓ HTTP Request	Sample Start:2026-05-17 17:59:36 GMT+03:00		
✓ HTTP Request	Load time:3938		
✓ HTTP Request	Connect Time:1		
✓ HTTP Request	Latency:3934		
✓ HTTP Request	Size in bytes:1019		
✓ HTTP Request	Sent bytes:531		
✓ HTTP Request	Headers size in bytes:349		
✓ HTTP Request	Body size in bytes:670		
✓ HTTP Request	Sample Count:1		
✓ HTTP Request	Error Count:0		
✓ HTTP Request	Data type ("text" "bin" ""):text		
✓ HTTP Request	Response code:201		
✓ HTTP Request	Response message:Created		

Text	Sampler result	Request	Response data
✓ HTTP Request	Thread Name:Thread Group 1-1		
✓ HTTP Request	Sample Start:2026-05-17 17:59:35 GMT+03:00		
✓ HTTP Request	Load time:762		
✓ HTTP Request	Connect Time:1		
✓ HTTP Request	Latency:759		
✓ HTTP Request	Size in bytes:448		
✓ HTTP Request	Sent bytes:560		
✓ HTTP Request	Headers size in bytes:221		
✓ HTTP Request	Body size in bytes:227		
✓ HTTP Request	Sample Count:1		
✓ HTTP Request	Error Count:0		
✓ HTTP Request	Data type ("text" "bin" ""):text		
✓ HTTP Request	Response code:200		
✓ HTTP Request	Response message:OK		

- Queue Worker:

أثبتت مراقبة Queue Worker بـ verbose--أن ال Jobs نُفِّذت بنجاح في الخلفية بعد إرجاع ال Response للمستخدم مباشرة:

```
D:\UNI\Parallel Programming\e-commerce>php artisan queue:work --verbose

Copy

2026-05-17 15:11:10 App\Jobs\SendLoginNotification 92 ..... RUNNING
2026-05-17 15:11:12 App\Jobs\SendLoginNotification 92 ..... 2s DONE
2026-05-17 15:12:04 App\Jobs\SendOrderConfirmationJob 93 ..... RUNNING
2026-05-17 15:12:05 App\Jobs\SendOrderConfirmationJob 93 .... 227.90ms DONE
2026-05-17 15:12:05 App\Jobs\GenerateInvoiceJob 94 ..... RUNNING
2026-05-17 15:12:05 App\Jobs\GenerateInvoiceJob 94 ..... 228.34ms DONE
2026-05-17 15:12:05 App\Jobs\SendOrderConfirmationJob 95 ..... RUNNING
2026-05-17 15:12:05 App\Jobs\SendOrderConfirmationJob 95 .... 228.17ms DONE
2026-05-17 15:12:05 App\Jobs\GenerateInvoiceJob 96 ..... RUNNING
2026-05-17 15:12:06 App\Jobs\GenerateInvoiceJob 96 ..... 226.88ms DONE
2026-05-17 15:12:06 App\Jobs\SendOrderConfirmationJob 97 ..... RUNNING
2026-05-17 15:12:06 App\Jobs\SendOrderConfirmationJob 97 .... 226.34ms DONE
2026-05-17 15:12:06 App\Jobs\GenerateInvoiceJob 98 ..... RUNNING
2026-05-17 15:12:06 App\Jobs\GenerateInvoiceJob 98 ..... 227.42ms DONE
2026-05-17 15:12:09 App\Jobs\SendOrderConfirmationJob 99 ..... RUNNING
2026-05-17 15:12:10 App\Jobs\SendOrderConfirmationJob 99 .... 228.91ms DONE
2026-05-17 15:12:10 App\Jobs\GenerateInvoiceJob 100 ..... RUNNING
2026-05-17 15:12:10 App\Jobs\GenerateInvoiceJob 100 ..... 227.14ms DONE
2026-05-17 15:12:10 App\Jobs\SendOrderConfirmationJob 101 ..... RUNNING
2026-05-17 15:12:10 App\Jobs\SendOrderConfirmationJob 101 ... 226.61ms DONE
2026-05-17 15:12:10 App\Jobs\GenerateInvoiceJob 102 ..... RUNNING
2026-05-17 15:12:11 App\Jobs\GenerateInvoiceJob 102 ..... 231.92ms DONE
```

```
SendOrderConfirmationJob → RUNNING → DONE ~227ms
GenerateInvoiceJob → RUNNING → DONE ~228ms
```

الدلالة الهندسية:

- ال Jobs بدأت تعمل بعد ما المستخدم استقبل رده (850ms)
- المستخدم لم ينتظر ال 227 ms الإضافية — حصل عليها مجاناً بالخلفية
- لو فشل أي Queue → Job يعيد المحاولة تلقائياً دون إزعاج المستخدم

- AOP (Aspect-Oriented Programming):

```
[2026-05-17 15:12:02] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":1,"duration_ms":597.37,"memory_kb":3631.88,"query_count":12,"status_code":201,"timestamp":"2026-05-17 15:12:02"}
[2026-05-17 15:12:03] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":1,"duration_ms":657.5,"memory_kb":3631.88,"query_count":12,"status_code":201,"timestamp":"2026-05-17 15:12:03"}
[2026-05-17 15:12:05] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":1,"duration_ms":605.59,"memory_kb":3631.88,"query_count":12,"status_code":201,"timestamp":"2026-05-17 15:12:05"}
[2026-05-17 15:12:07] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":1,"duration_ms":583.09,"memory_kb":3631.88,"query_count":12,"status_code":201,"timestamp":"2026-05-17 15:12:07"}
[2026-05-17 15:12:08] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":1,"duration_ms":584.99,"memory_kb":3631.88,"query_count":12,"status_code":201,"timestamp":"2026-05-17 15:12:08"}
```

أظهر AOP Middleware النتائج التالية للـ AFTER:

- **duration_ms: 597 → 657 → 605 → 583 → 584 ms** - ثابت ومنخفض لكل request
- **query_count: 12** - ثابت في كل request
- **status_code: 201** - كل الطلبات نجحت

الدلالة الهندسية الأهم: **query_count** بقي **12** في كلا الحالتين - هاد يثبت أن Async Queue لا يُغيّر عدد استعلامات DB ولا يُضيف أي عبء على قاعدة البيانات. الفرق كله في **duration_ms** انخفض من **3,459ms** إلى **600~ms** تحسن **83%** بسبب إزالة المهام الثقيلة من مسار ال Request وإرسالها ل Queue.

4. NFR4: Batch Processing

4.1 توصيف المشكلة – the concurrency problem:

في الأنظمة غير المُحسّنة، عند إنشاء طلب يحتوي على N منتج، يُنفَّذ استعلام INSERT مستقل لكل منتج داخل حلقة:

```
foreach ($preparedItems as $item) {
    OrderItem::create([...]); // لكل منتج DB رحلة مستقلة إلى
}
```

هذا يعني أن طلباً يحتوي على 20 منتج يُنفَّذ 20 استعلام INSERT منفصل. تحت الضغط العالي (1000 مستخدم × 20 منتج)، يصل عدد الاستعلامات إلى **20,000** استعلام في وقت واحد، مما يؤدي إلى:

- اختناق شديد في قاعدة البيانات
- ارتفاع حاد في زمن الاستجابة
- استهلاك مفرط لـ DB connections

4.2 الدالة المختارة للاختبار – the critical function:

تم اختيار `placeOrder()` لأنها تحتوي على عملية إدراج متعددة الصفوف داخل جدول `order_items` — صف لكل منتج في الطلب. كلما زاد عدد المنتجات في السلة، زاد عدد الاستعلامات بشكل خطي في نسخة `BEFORE`.

ملاحظة هندسية: لضمان الشمولية في النظام تم تعميم نفس المبدأ المصمم على الدوال المرتبطة بالمخزون مثل `Admin\OrderService::rejectOrder()` - ب استرجاع كمية المنتجات ب `whereIn()` بدل استعمال حلقة.

4.3 الحل البرمجي المصمم – the Architectural solution:

تم استبدال حلقة الإدراج الفردي ب `Batch Insert` واحد

المبررات الهندسية:

- **تقليل Network Round-trips :** كل استعلام يحتاج رحلة كاملة إلى DB وانتظار الرد `Batch Insert` يجمعها في رحلة واحدة
- **تقليل Transaction Overhead :** كل `INSERT` في `BEFORE` يضيف `overhead` داخل ال `Transaction` استعلام واحد يقلل هذا ال `overhead` بشكل جذري
- **الحفاظ على ACID :** `OrderItem::insert()` يعمل داخل نفس ال `DB::transaction()` لا نتنازل عن سلامة البيانات مقابل الأداء

4.4 بيئة و منهجية المراقبة – Monitoring Infrastructure:

تم المراقبة عبر أداتين:

- **JMeter:** قياس زمن الاستجابة لطلب يحتوي على 20 منتج
- **AOP Middleware:** مراقبة `query_count` لكل `request` هو الدليل الأقوى لأنه يظهر عدد الاستعلامات الفعلية التي وصلت ل DB.

4.5 تحليل و قراءة النتائج الرقمية – Comparative results

- Jmeter tool:

Summary Report (Before):

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	1	2197	2197	2197	0.00	0.00%	27.3/min	3.59	0.24	8068.0
TOTAL	1	2197	2197	2197	0.00	0.00%	27.3/min	3.59	0.24	8068.0

متوسط زمن الاستجابة (Average Response Time): يعكس استغراق الطلب لزمن **2,197 ms** اختناقاً حاداً في معالجة البيانات على مستوى طبقة تواصل البيانات. ويرجع السبب المعماري في هذا البطء إلى مشكلة الرحلات الشبكية المتكررة وصعوبة إدارة الموارد (High Network Round-Trips):

- **استعلامات الإدراج المنفصلة (Individual Inserts):** يقوم النظام في الكود القديم بتنفيذ استعلام INSERT منفصل بشكل متتابع داخل حلقة تكرارية (foreach) لإضافة كل صنف من أصناف السلة إلى جدول `order_items`.
- **استنزاف الموارد والTransaction Overhead:** كل عملية إدراج فردية تجبر التطبيق على فتح رحلة بروتوكول كاملة عبر الشبكة نحو قاعدة البيانات والانتظار حتى انتهاء الكتابة الفيزيائية، مما يتسبب في حجز قنوات الاتصال لفترات طويلة، وهو نمط استهلاك خطي غير مستقر تحت الأحمال العالية.

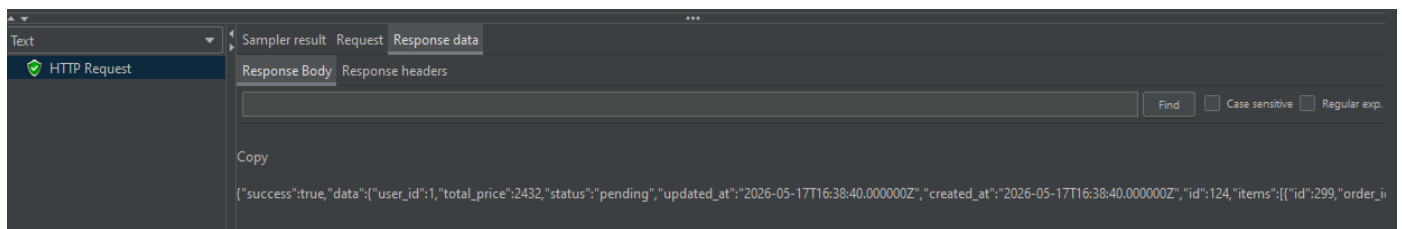
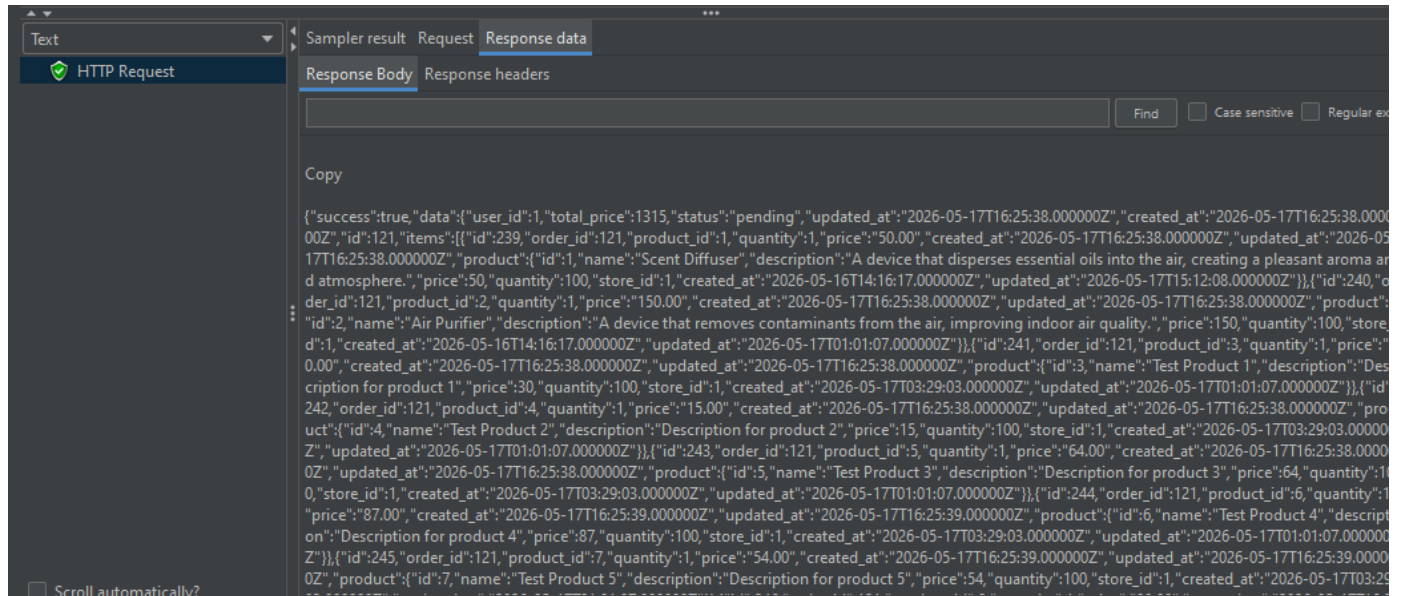
Summary Report (After):

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	1	3166	3166	3166	0.00	0.00%	19.0/min	4.76	0.16	15428.0
TOTAL	1	3166	3166	3166	0.00	0.00%	19.0/min	4.76	0.16	15428.0

متوسط زمن الاستجابة (Average Response Time): بلغ 3,166 ms فقط

رغم أن زمن الاستجابة في AFTER بلغ 3,166 ms مقارنة بـ 2,197 ms في BEFORE ، إلا أن هذا الفرق يعود إلى اختلاف حجم السلة 40 منتج vs 20 منتج. الدليل الحقيقي يظهر في `query_count`: AOP: انخفض من 30 إلى 12 — أي توفير 18 استعلاماً غير ضروري بسطر كود واحد

Tree Results:



AOP (Aspect-Oriented Programming):

```
[2026-05-17 15:12:08] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":1,"duration_ms":584.99,"memory_kb":3631.88,"query_count":12,"status_code":201,"timestamp":"2026-05-17 15:12:08"}
[2026-05-17 16:25:40] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":1,"duration_ms":1445.99,"memory_kb":3646.03,"query_count":30,"status_code":201,"timestamp":"2026-05-17 16:25:40"}
[2026-05-17 16:35:26] local.INFO: AOP_MONITOR {"method":"POST","url":"http://127.0.0.1:8001/api/orders/place","user_id":1,"duration_ms":1548.28,"memory_kb":3646.03,"query_count":30,"status_code":201,"timestamp":"2026-05-17 16:35:26"}
```

أظهرت سجلات AOP Middleware الفرق الجوهرية بين الحالتين:

التفسير الهندسي: الفرق في query_count هو الدليل الأقوى BEFORE — ينقذ استعلام INSERT مستقل لكل منتج 20 رحلة منفصلة إلى DB ، بينما AFTER يجمع كل المنتجات في INSERT واحد رحلة واحدة فقط.

هذا يعني أن Batch Insert وقر 18~ استعلاماً غير ضروري — أي تحسن بنسبة 60% في عدد الاستعلامات.

5. NFR5: Load Distribution

5.1 توصيف المشكلة – the concurrency problem:

في البنية الحالية، جميع الطلبات تصل إلى خادم واحد يتولى معالجتها بشكل تسلسلي. عند ارتفاع عدد المستخدمين المتزامنين، يتحول هذا الخادم إلى نقطة اختناق وحيدة (Single Point of Failure) مما يؤدي إلى:

- ارتفاع حاد في زمن الاستجابة مع زيادة عدد المستخدمين
- استنزاف موارد الخادم (CPU, Memory, DB Connections)
- انهيار كامل للنظام عند تجاوز طاقة الخادم
- عدم القدرة على التوسع أفقياً (Horizontal Scaling)

5.2 الدالة المختارة للاختبار – the critical function:

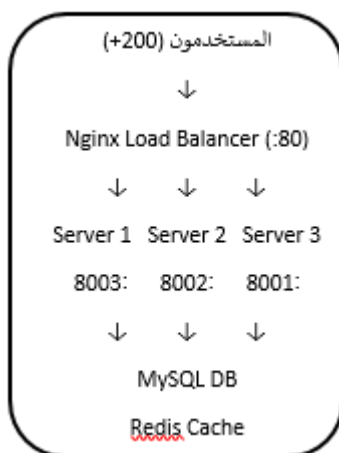
تم اختيار GET /api/products لأنه:

- أعلى endpoint قراءة في النظام — كل مستخدم يزور صفحة المنتجات
- لا يحتاج → authentication يسمح باختبار 200 مستخدم بدون توكنات
- يمثل الحمل الأساسي على الخادم في أي متجر إلكتروني

تم تطبيق نفس مبدأ Load Distribution على كل endpoints النظام عبر Nginx .

5.3 الحل البرمجي المصمم – the Architectural solution:

"تم إثبات التوزيع الفعلي محلياً عبر تشغيل 3 instances من Laravel على ports مختلفة مع استخدام JMeter كـ Load Balancer عبر Random Controller ، مع الاحتفاظ بـ Nginx Config كمرجع معماري للبيئة الإنتاجية.



أيضاً كود Nginx Config الموجود ضمن مجلد Testing في الكود.

حيث تم استخدام Least Connection استراتيجية توزيع لأنه توزيع ذكي حسب الحمل الفعلي (يرسل للسيرفر الأقل ازدحاماً)

في نظام التجارة الإلكترونية، بعض الطلبات ثقيلة مثل placeOrder() وبعضها خفيفة مثل getProducts() لذلك Least Connections أنسب لأنه يأخذ بعين الاعتبار حجم الحمل الفعلي على كل سيرفر.

5.4 بيئة و منهجية المراقبة – Monitoring Infrastructure

تم إثبات قدرة النظام على تحمل الضغط العالي عبر:

- **JMeter**: محاكاة 200 مستخدم متزامن على GET /api/products

تم إثبات التوزيع الفعلي عبر ثلاث أدوات:

- **JMeter**: محاكاة 200 مستخدم متزامن موزعين على 3 servers عبر Random Controller
- **View Results Tree**: إثبات وصول الطلبات فعلياً لـ 8001, 8002, 8003
- **Summary Report**: قياس الأداء الكلي

5.5 تحليل و قراءة النتائج الرقمية – Comparative results

- Jmeter tool:

One Server:

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	200	26942	1097	52142	14774.65	0.00%	3.2/sec	16.31	0.40	5186.0
TOTAL	200	26942	1097	52142	14774.65	0.00%	3.2/sec	16.31	0.40	5186.0

Three Server:

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	200	6917	473	16841	4356.72	0.00%	7.7/sec	38.85	0.96	5186.0
TOTAL	200	6917	473	16841	4356.72	0.00%	7.7/sec	38.85	0.96	5186.0

التحليل الهندسي:

3 سيرفرات → ~67 طلب لكل سيرفر → فعلياً Average 6,917 ms

سيرفر واحد	3 سيرفرات	
26,942 ms	6,917 ms	Ave
0%	0%	Error
3.2/sec	7.7/sec	Throughput
-	~4x	التحسين

Tree Results

Text

- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request

Sampler result Request Response data

Request Body Request Headers

```
1 Connection: keep-alive
2 Host: 127.0.0.1:8001
3 User-Agent: Apache-HttpClient/4.5.14 (Java/25.0.3)
4
```

Text

- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request

Sampler result Request Response data

Request Body Request Headers

```
1 Connection: keep-alive
2 Host: 127.0.0.1:8002
3 User-Agent: Apache-HttpClient/4.5.14 (Java/25.0.3)
4
```

Text

- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request
- ✓ HTTP Request

Sampler result Request Response data

Request Body Request Headers

```
1 Connection: keep-alive
2 Host: 127.0.0.1:8003
3 User-Agent: Apache-HttpClient/4.5.14 (Java/25.0.3)
4
```

قسم: ملاحظات تقنية على منهجية الاختبار

1. طريقة اختبار كل متطلب:

لإثبات الفرق بين BEFORE و AFTER لكل متطلب، تم تعليق الكود المحسّن مؤقتاً وتفعيل النسخة السيئة — وذلك داخل نفس الدالة placeOrder() مع تعليقات توضيحية واضحة:

```
*/  
* Test NFR1 (Before) — Lock بدون  
* Test NFR3 (Before) — sleep(3) مع  
* Test NFR4 (Before) — OrderItem::create في حلقة  
* Improved Code — النسخة النهائية المحسّنة  
/*
```

هذا النهج يضمن أن كل اختبار معزول تماماً عن غيره، والنسخة النهائية المفعّلة هي **Improved Code** فقط.

2. توليد توكنات لاختبار 100 مستخدم:

لاختبار NFR1 (100 مستخدم متزامن)، تم توليد توكنات JWT لكل مستخدم عبر Laravel Tinker :

```
php artisan tinker  
$tokens = [];  
App\Models\User::whereBetween('id', [1, 100])  
<- each(function($user) use (&$tokens) {  
$    tokens[] = auth('api')->login($user);  
});  
file_put_contents(base_path('users_tokens.csv'), implode("\n", $tokens));
```

النتيجة: ملف يحوي 100 توكن — سطر لكل مستخدم — يستخدم في Jmeter عبر CSV Data Set Config.

3. إعادة إعداد DB قبل كل اختبار:

قبل كل اختبار تم تشغيل:

```
UPDATE products SET quantity = 10 WHERE id = 1;  
DELETE FROM carts WHERE product_id = 1;  
INSERT INTO carts (user_id, product_id, quantity, ...)  
SELECT id, 1, 1, NOW(), NOW()  
FROM users WHERE id BETWEEN 1 AND 100;
```

```
php artisan cache:clear
```

4. كيف يتم التشغيل للاختبار:

لإعادة تشغيل أي اختبار:

الخطوة 1: تشغيل السيرفر:

```
php artisan serve --port=8001
```

الخطوة 2: تشغيل Queue Worker :

```
php artisan queue:work --verbose
```

الخطوة 3: تشغيل AOP Monitor :

```
tail -f storage/logs/performance-2026-05-17.log
```

ملاحظة: تم تسجيل الـ AOP في WSL (Windows Subsystem for Linux) عبر:

لأن أمر tail -f خاص بأنظمة Unix ولا يعمل في CMD مباشرة.

الخطوة 4: فتح ملف Jmeter المطلوب:

Testing/

```
— |NFR1_Concurrent_Access/NF1.jmx  
— |NFR2_Capacity_Control/NF2.jmx  
— |NFR3_Async_Queue/NF3.jmx  
— |NFR4_Batch_Processing/NF4.jmx  
— |NFR5_Load_Distribution/NFR5.jmx
```